



数字信号处理实验报告

实验内容: 实验二: IIR 数字滤波器设计及软件实现

专 业: 工科试验班 (未来空天领军计划)

班 级: 242311 班

姓 名: 傅梁训

学 号: 24373088

任课教师: 白桐老师

2026 年 6 月

实验二：IIR 数字滤波器设计及软件实现

学 生： 傅梁训

指导老师： 白桐老师

摘 要

本实验围绕 IIR 数字滤波器设计与软件实现展开。实验首先产生三路抑制载波单频调幅信号，并将其相加得到复合信号。三路信号在时域叠加后难以直接分辨，但其频率分量分别位于 225 Hz、275 Hz、450 Hz、550 Hz、900 Hz 和 1100 Hz，在频域中互不重叠，因此可以通过低通、带通和高通滤波器进行分离。

程序使用 Python 信号处理函数设计三个椭圆 IIR 数字滤波器，通带最大衰减取 $R_p = 0.1$ dB，阻带最小衰减取 $R_s = 60$ dB。实验结果显示，低通、带通和高通滤波器阶数分别为 7、6、9，均满足通阻带指标。滤波输出的频谱表明，三个滤波器能够分别保留目标频段并抑制其他信号成分，从而实现复合调幅信号的频域分离。

关键词：IIR 数字滤波器；椭圆滤波器；抑制载波调幅；频域分离；FFT

Experiment 2: IIR Digital Filter Design and Software Implementation

Author: Fu Liangxun

Tutor: Bai Tong

Abstract

This experiment focuses on the design and software implementation of IIR digital filters. Three suppressed-carrier amplitude-modulated signals are generated and summed into a composite signal. Although the components are mixed in the time domain, their spectral lines are separated in the frequency domain, which makes filter-based separation feasible.

Three elliptic IIR filters are designed in Python, including a low-pass filter, a band-pass filter, and a high-pass filter. The maximum passband loss is 0.1 dB, and the minimum stopband attenuation is 60 dB. The obtained filter orders are 7, 6, and 9, respectively. The filtering results show that the desired spectral components are preserved while the other components are suppressed, verifying the effectiveness of frequency-domain separation using IIR filters.

Key words: IIR filter; elliptic filter; SCAM; frequency separation; FFT

目 录

图目录	IV
表目录	V
1 实验目的	1
2 实验原理	2
2.1 抑制载波调幅信号的频谱	2
2.2 椭圆 IIR 数字滤波器	3
3 实验内容与结果分析	4
3.1 主程序框图	4
3.2 复合信号的时域波形与频谱	5
3.3 滤波器损耗函数	5
3.4 滤波分离结果	6
4 实验结论	8
5 思考题	9
5.1 确定三路调幅信号的载波频率和调制信号频率	9
5.2 改变采样点数时是否能得到 6 根理想谱线	9
5.3 加入载波成分后的 AM 信号与 SCAM 信号差别	10
附录 A 程序清单	12

插图目录

图 3.1 实验二主程序框图	4
图 3.2 复合信号 $s(t)$ 的时域波形和幅频特性	5
图 3.3 三个椭圆分离滤波器的损耗函数曲线	6
图 3.4 三路滤波输出信号的时域波形和幅频特性	7
图 5.1 不同采样点数下的 FFT 频谱对比	10
图 5.2 AM 信号与抑制载波调幅信号的时域和频域对比	11

表格目录

表 2.1 三路抑制载波调幅信号参数	2
表 2.2 三个椭圆滤波器的设计指标	3

1 实验目的

本实验围绕 IIR 数字滤波器设计与软件实现展开。通过调用 Python 信号处理函数设计低通、带通和高通三个椭圆数字滤波器，将由三路抑制载波调幅信号相加得到的复合信号分离出来。实验中需要观察复合信号和滤波输出的时域波形及频谱，理解频域分离的基本思想，并掌握根据通带、阻带指标设计 IIR 数字滤波器的方法。

2 实验原理

2.1 抑制载波调幅信号的频谱

一路抑制载波单频调幅信号可写为

$$s(t) = \cos(2\pi f_m t) \cos(2\pi f_c t), \quad (2.1)$$

其中， f_c 为载波频率， f_m 为调制信号频率。利用积化和差公式可得

$$s(t) = \frac{1}{2} \cos [2\pi(f_c - f_m)t] + \frac{1}{2} \cos [2\pi(f_c + f_m)t]. \quad (2.2)$$

因此一路抑制载波调幅信号只有两个频率分量，分别位于 $f_c - f_m$ 和 $f_c + f_m$ ，并且关于载波频率 f_c 对称。三路信号在时域相加后互相混叠，但只要频谱互不重叠，就可以通过滤波器在频域将其分离。

本实验采样频率和采样点数为

$$F_s = 10000 \text{ Hz}, \quad N = 1600. \quad (2.3)$$

三路信号参数如表 2.1 所示。

表 2.1 三路抑制载波调幅信号参数

信号	载波频率 f_c/Hz	调制频率 f_m/Hz	差频/Hz	和频/Hz
$x_1(t)$	1000	100	900	1100
$x_2(t)$	500	50	450	550
$x_3(t)$	250	25	225	275

2.2 椭圆 IIR 数字滤波器

IIR 数字滤波器的一般系统函数为

$$H(z) = \frac{B(z)}{A(z)} = \frac{b_0 + b_1 z^{-1} + \cdots + b_M z^{-M}}{1 + a_1 z^{-1} + \cdots + a_N z^{-N}}. \quad (2.4)$$

椭圆滤波器在通带和阻带均具有等波纹特性，在相同通阻带指标下通常能以较低阶数满足设计要求。本实验要求通带最大衰减为 $R_p = 0.1 \text{ dB}$ ，阻带最小衰减为 $R_s = 60 \text{ dB}$ 。若定义滤波器损耗函数为

$$A(f) = -20 \log_{10} \left| H(e^{j2\pi f/F_s}) \right|, \quad (2.5)$$

则通带和阻带指标分别表示为

$$A(f) \leq R_p, \quad f \in \Omega_p, \quad A(f) \geq R_s, \quad f \in \Omega_s. \quad (2.6)$$

根据复合信号的频谱位置，三个滤波器的通带和阻带频率选取如表 2.2 所示。程序中使用 `scipy.signal.ellipord` 求满足指标的最低阶数，再使用 `scipy.signal.ellip` 得到二阶节形式的 IIR 滤波器，并用 `sosfilt` 进行滤波实现。

表 2.2 三个椭圆滤波器的设计指标

滤波器	分离信号	通带频率/Hz	阻带频率/Hz	阶数
低通	$y_3(n)$	0 ~ 300	400 ~ 5000	7
带通	$y_2(n)$	400 ~ 600	0 ~ 350, 700 ~ 5000	6
高通	$y_1(n)$	800 ~ 5000	0 ~ 700	9

3 实验内容与结果分析

3.1 主程序框图

实验主程序流程如图 3.1 所示。程序先产生三路抑制载波调幅信号并相加得到 $s(t)$ ，然后根据频谱分布分别设计低通、带通和高通椭圆滤波器，最后对复合信号滤波并绘制输出信号的时域和频域结果。

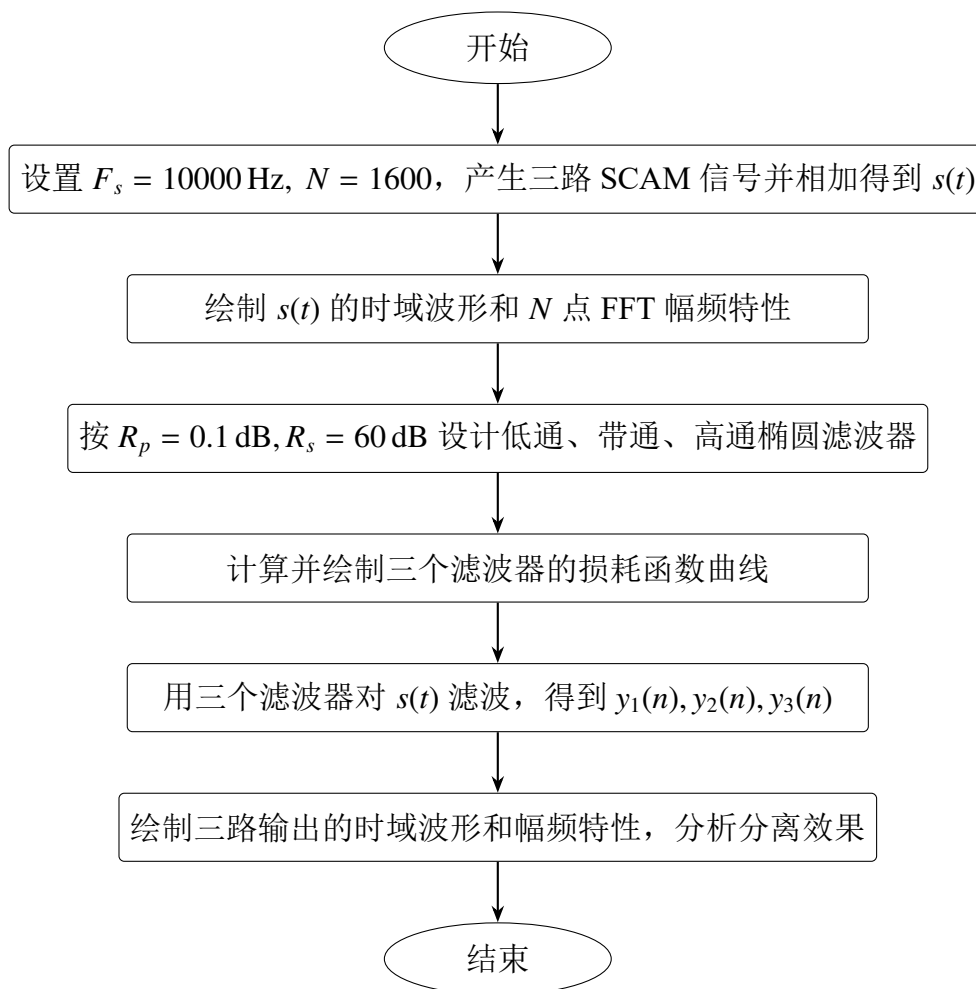


图 3.1 实验二主程序框图

3.2 复合信号的时域波形与频谱

由三路信号相加得到的复合信号为

$$s(t) = x_1(t) + x_2(t) + x_3(t). \quad (3.1)$$

其时域波形和单边幅频特性如图 3.2 所示。可以看到，时域波形中三路调幅信号已经叠加在一起，不能直接分辨；但频谱上出现 225 Hz、275 Hz、450 Hz、550 Hz、900 Hz、1100 Hz 六根谱线，三路信号在频域是分开的，因此可以用滤波器进行分离。

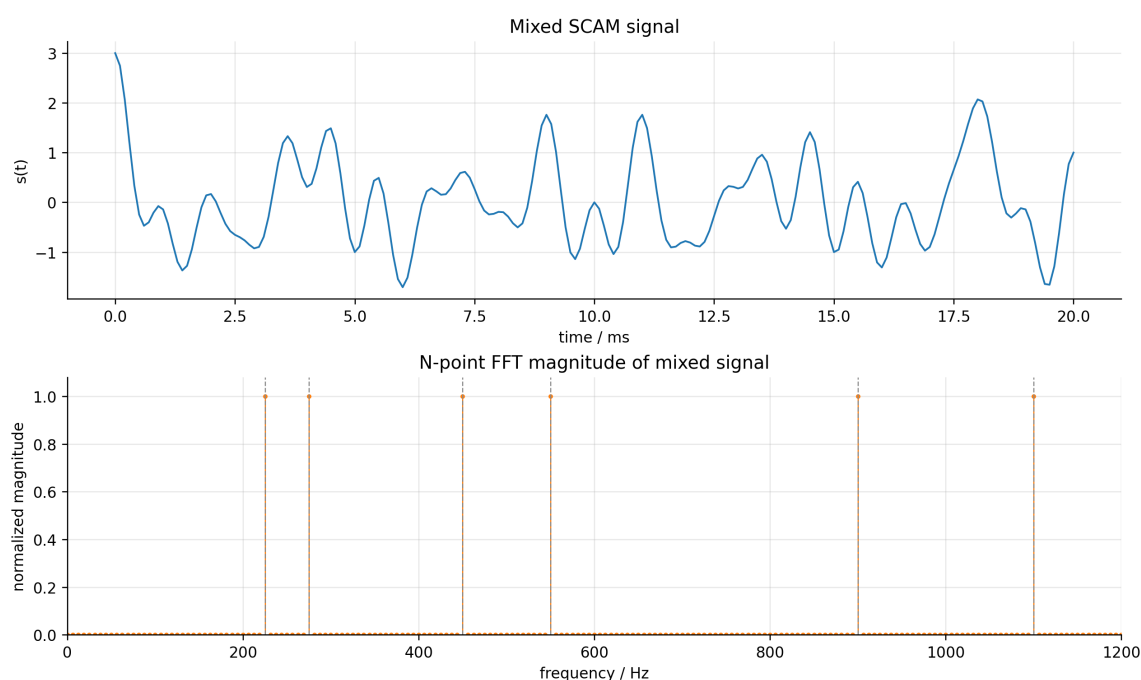


图 3.2 复合信号 $s(t)$ 的时域波形和幅频特性

3.3 滤波器损耗函数

三个滤波器的损耗函数如图 3.3 所示。程序计算得到低通、带通、高通滤波器的阶数分别为 7、6、9；在各自通带内最大损耗均为 0.1000 dB，在阻带内最小衰减均达到 60.0000 dB。这说明所设计的椭圆滤波器满足题目要求。

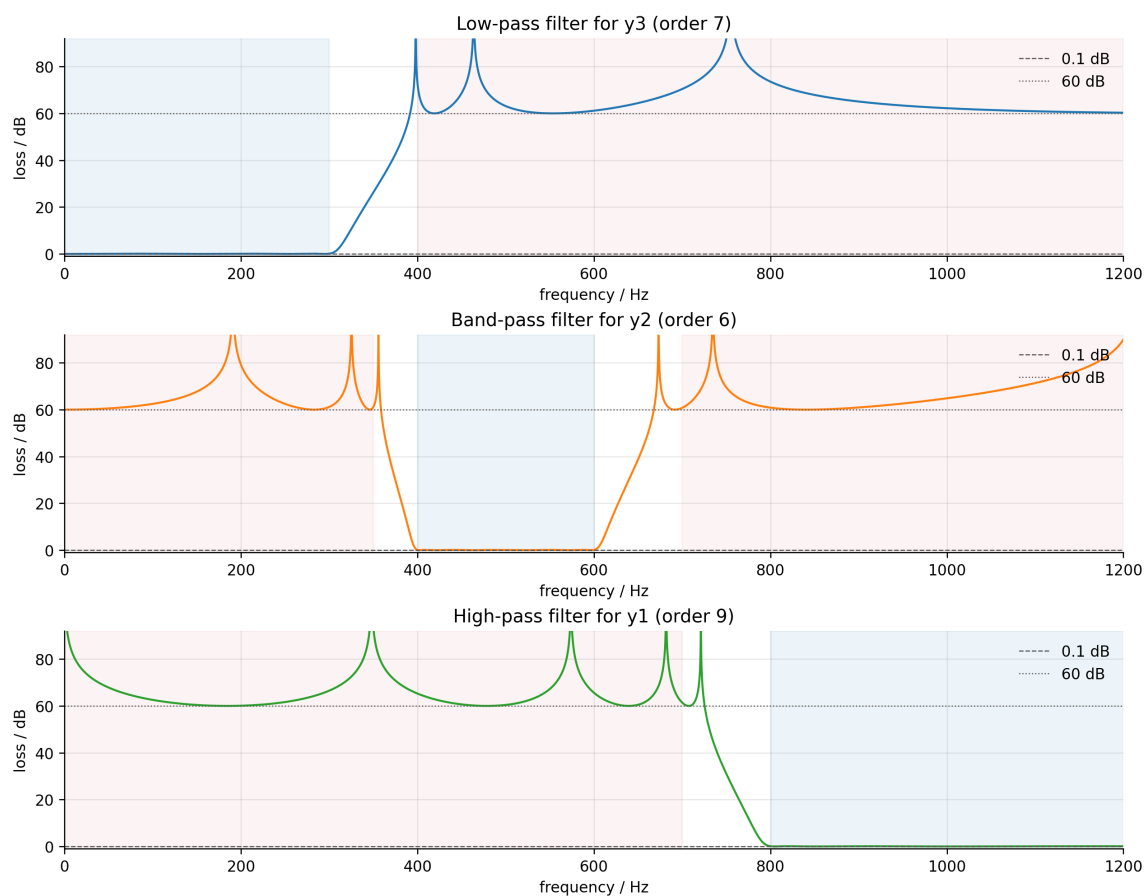


图 3.3 三个椭圆分离滤波器的损耗函数曲线

3.4 滤波分离结果

使用三个滤波器分别对复合信号 $s(t)$ 进行滤波，分离结果如图 3.4 所示。由于 IIR 滤波器在起始阶段存在瞬态响应，图中时域波形取 20 ms 以后的一段进行观察。低通滤波器保留 225 Hz 和 275 Hz 附近的谱线，对应 $f_c = 250$ Hz 的第三路信号；带通滤波器保留 450 Hz 和 550 Hz 附近的谱线，对应 $f_c = 500$ Hz 的第二路信号；高通滤波器保留 900 Hz 和 1100 Hz 附近的谱线，对应 $f_c = 1000$ Hz 的第一路信号。由频谱结果可以看出，三路信号被较好分离。

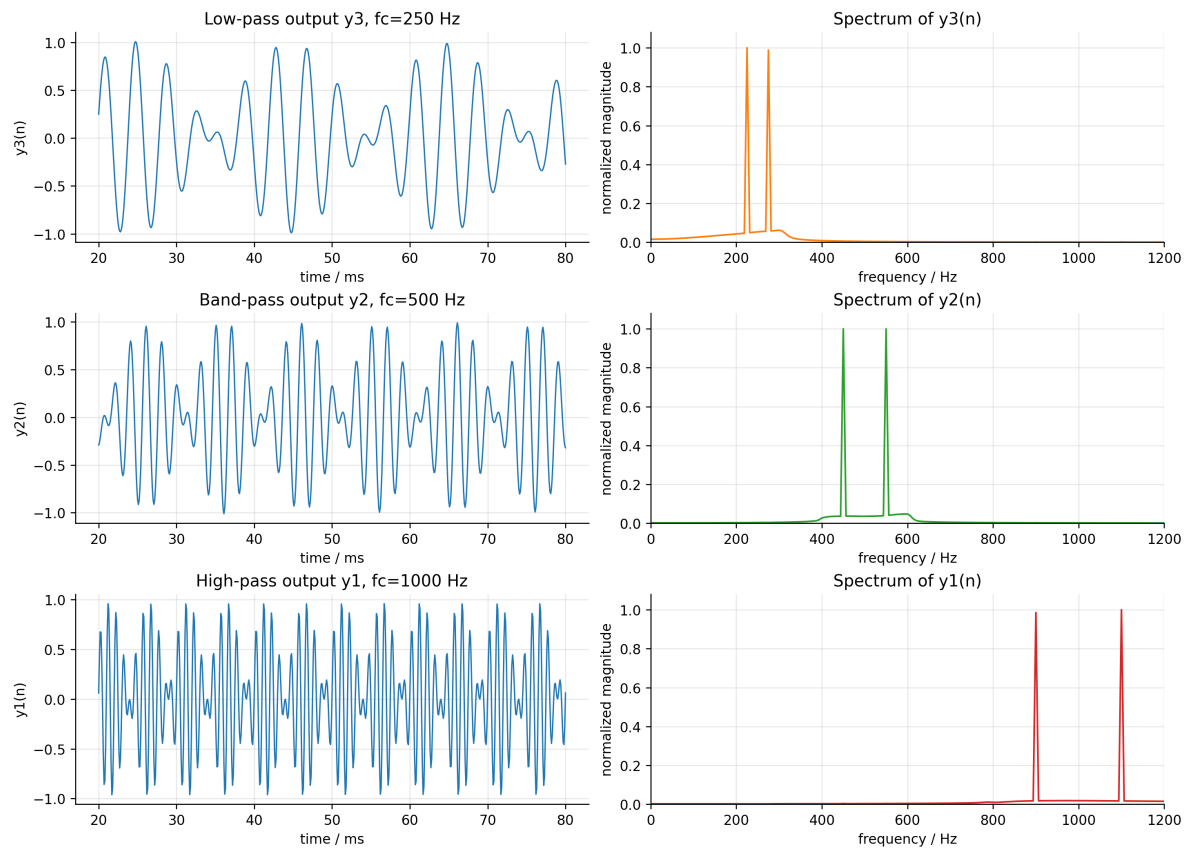


图 3.4 三路滤波输出信号的时域波形和幅频特性

4 实验结论

本实验通过椭圆 IIR 数字滤波器实现了三路抑制载波调幅信号的频域分离。实验结果说明：虽然三路信号在时域相加后难以直接分辨，但其频率成分互不重叠时，可以根据频谱位置设计不同类型的滤波器进行分离。椭圆滤波器在满足 0.1 dB 通带衰减和 60 dB 阻带衰减的条件下阶数较低，适合完成这种过渡带较窄的分离任务。实际软件实现中采用二阶节形式可以提高 IIR 滤波器的数值稳定性。

5 思考题

5.1 确定三路调幅信号的载波频率和调制信号频率

由信号产生函数可知：

$$f_{c1} = \frac{F_s}{10} = 1000 \text{ Hz}, \quad f_{m1} = \frac{f_{c1}}{10} = 100 \text{ Hz}; \quad (5.1)$$

$$f_{c2} = \frac{F_s}{20} = 500 \text{ Hz}, \quad f_{m2} = \frac{f_{c2}}{10} = 50 \text{ Hz}; \quad (5.2)$$

$$f_{c3} = \frac{F_s}{40} = 250 \text{ Hz}, \quad f_{m3} = \frac{f_{c3}}{10} = 25 \text{ Hz}. \quad (5.3)$$

因此三路抑制载波调幅信号的正频率谱线分别为 900 Hz、1100 Hz、450 Hz、550 Hz，以及 225 Hz、275 Hz。

5.2 改变采样点数时是否能得到 6 根理想谱线

对长度为 N 的序列作 N 点 FFT 时，频率分辨率为

$$\Delta f = \frac{F_s}{N}. \quad (5.4)$$

若某一频率 f 正好落在 DFT 栅格上，则需满足

$$k = \frac{fN}{F_s} \quad (5.5)$$

为整数。否则该频率不能只集中在一个 DFT 频点上，会出现频谱泄漏，因而不能得到理想谱线。

当 $N = 1600$ 时， $\Delta f = 6.25 \text{ Hz}$ ，六个频率 225, 275, 450, 550, 900, 1100 Hz 都落在整数频点上，所以能得到 6 根理想谱线。当 $N = 1800$ 时， $\Delta f = 5.5556 \text{ Hz}$ ，其中

$$\frac{225 \times 1800}{10000} = 40.5, \quad \frac{275 \times 1800}{10000} = 49.5, \quad (5.6)$$

不是整数，因此不能得到全部 6 根理想谱线。虽然 450 Hz、550 Hz、900 Hz、1100 Hz 仍落在整数频点上，但 225 Hz 和 275 Hz 会发生频谱泄漏。当 $N = 2000$ 时， $\Delta f = 5$ Hz，六个频率均为 5 Hz 的整数倍，所以又可以得到 6 根理想谱线。程序验证结果如图 5.1 所示。

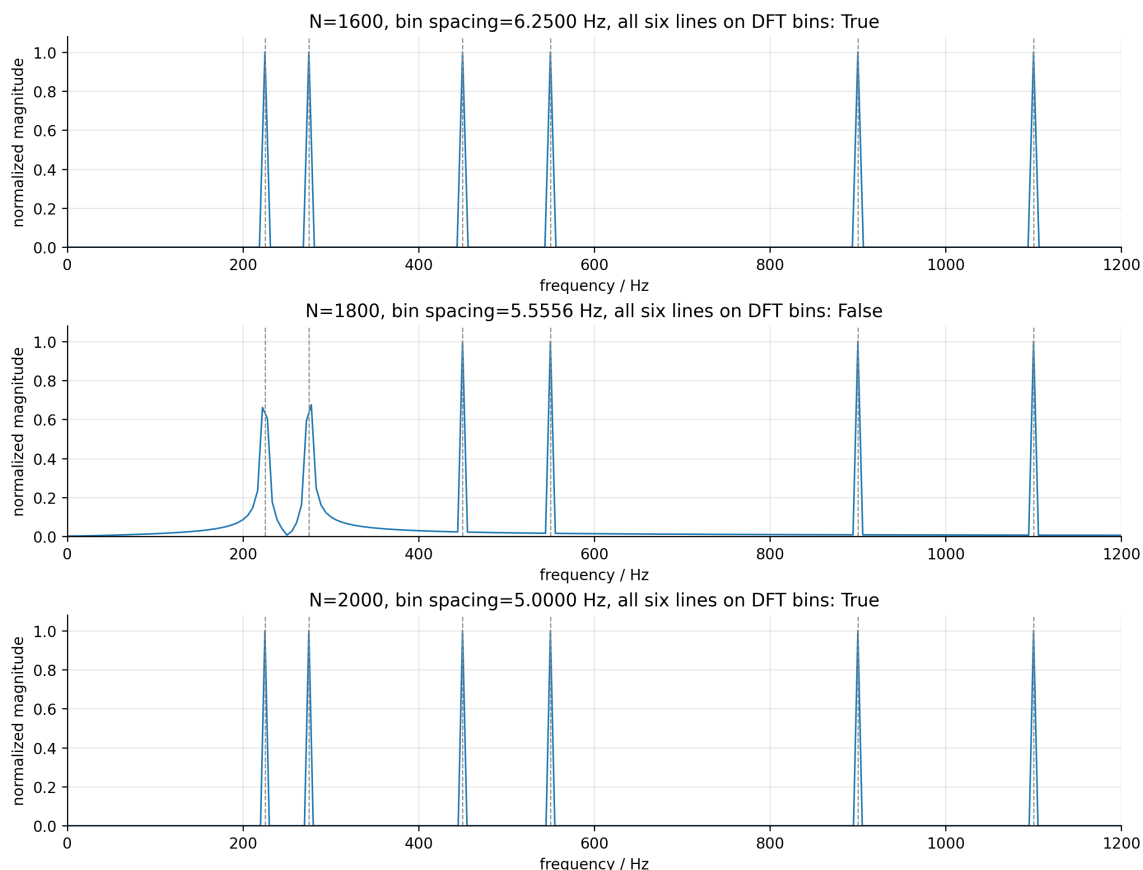


图 5.1 不同采样点数下的 FFT 频谱对比

5.3 加入载波成分后的 AM 信号与 SCAM 信号差别

将每路信号改为 AM 信号：

$$s_{AM}(t) = [A_0 + A_m \cos(2\pi f_m t)] \cos(2\pi f_c t), \quad (5.7)$$

本实验取 $A_0 = A_m = 1$ 。展开可得

$$s_{AM}(t) = A_0 \cos(2\pi f_c t) + \frac{A_m}{2} \cos[2\pi(f_c - f_m)t] + \frac{A_m}{2} \cos[2\pi(f_c + f_m)t]. \quad (5.8)$$

与抑制载波调幅相比，AM 信号除了和频、差频两根边带谱线外，还多出载波频率 f_c 处的谱线。因此本实验三路 AM 信号会在 250 Hz、500 Hz、1000 Hz 处增加明显载波分量。时域上，AM 信号的包络中含有直流偏置项，波形包络比抑制载波调幅更直观；而 SCAM 信号没有载波分量，频谱中只保留上下边带。对比结果如图 5.2 所示。

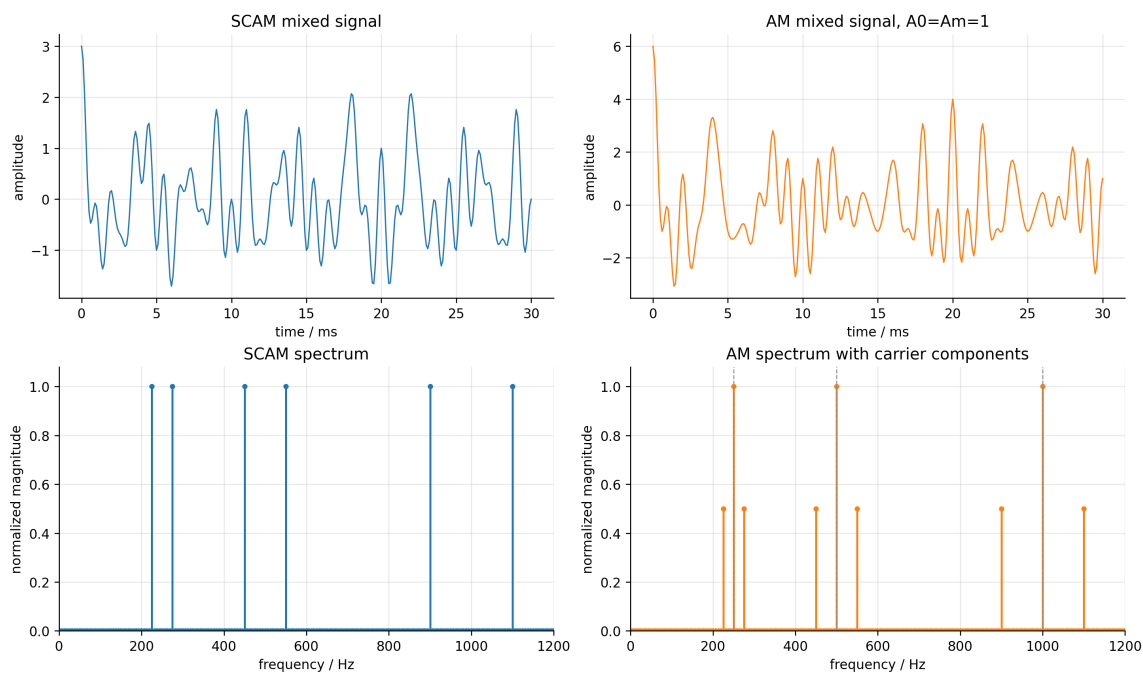


图 5.2 AM 信号与抑制载波调幅信号的时域和频域对比

附录 A 程序清单

```
1 from pathlib import Path
2
3 import matplotlib
4
5 matplotlib.use("Agg")
6
7 import matplotlib.pyplot as plt
8 import numpy as np
9 from scipy import signal
10
11
12 FS = 10000
13 N = 1600
14 RP = 0.1
15 RS = 60
16 OUT_DIR = Path(__file__).resolve().parent / "figures" / "experiment2"
17
18 COMPONENTS = [
19     {"name": "y1", "fc": 1000, "fm": 100, "label": "fc=1000 Hz, fm=100 Hz"},
20     {"name": "y2", "fc": 500, "fm": 50, "label": "fc=500 Hz, fm=50 Hz"},
21     {"name": "y3", "fc": 250, "fm": 25, "label": "fc=250 Hz, fm=25 Hz"},
22 ]
23
24 FILTER_SPECS = {
25     "lowpass_y3": {
26         "title": "Low-pass filter for y3",
27         "btype": "lowpass",
28         "wp": 300,
29         "ws": 400,
30         "passbands": [(0, 300)],
31         "stopbands": [(400, FS / 2)],
32         "target": "y3",
33     },
34     "bandpass_y2": {
35         "title": "Band-pass filter for y2",
36         "btype": "bandpass",
37         "wp": [400, 600],
38         "ws": [350, 700],
39         "passbands": [(400, 600)],
40         "stopbands": [(0, 350), (700, FS / 2)],
41         "target": "y2",
42     },
43     "highpass_y1": {
44         "title": "High-pass filter for y1",
```

```
45     "btype": "highpass",
46     "wp": 800,
47     "ws": 700,
48     "passbands": [(800, FS / 2)],
49     "stopbands": [(0, 700)],
50     "target": "y1",
51 },
52 }
53
54
55 def set_common_style():
56     plt.rcParams.update(
57         {
58             "figure.dpi": 120,
59             "savefig.dpi": 240,
60             "axes.grid": True,
61             "grid.alpha": 0.28,
62             "axes.spines.top": False,
63             "axes.spines.right": False,
64             "font.size": 10,
65             "legend.frameon": False,
66         }
67     )
68
69
70 def make_time_axis(n_samples=N):
71     return np.arange(n_samples) / FS
72
73
74 def scam_component(t, fc, fm):
75     return np.cos(2 * np.pi * fm * t) * np.cos(2 * np.pi * fc * t)
76
77
78 def am_component(t, fc, fm, a0=1.0, am=1.0):
79     return (a0 + am * np.cos(2 * np.pi * fm * t)) * np.cos(2 * np.pi * fc * t)
80
81
82 def build_signal(n_samples=N, am=False):
83     t = make_time_axis(n_samples)
84     components = {}
85     for item in COMPONENTS:
86         if am:
87             components[item["name"]] = am_component(t, item["fc"], item["fm"])
88         else:
89             components[item["name"]] = scam_component(t, item["fc"], item["fm"])
90     mixed = sum(components.values())
91     return t, mixed, components
92
```

```
93
94 def expected_lines():
95     lines = []
96     for item in COMPONENTS:
97         lines.extend([item["fc"] - item["fm"], item["fc"] + item["fm"]])
98     return sorted(lines)
99
100
101 def single_sided_spectrum(x, fs=FS, nfft=None, normalize=True):
102     if nfft is None:
103         nfft = len(x)
104     freq = np.fft.rfftfreq(nfft, d=1 / fs)
105     spectrum = np.fft.rfft(x, nfft)
106     mag = np.abs(spectrum)
107     if normalize and mag.max() > 0:
108         mag = mag / mag.max()
109     return freq, mag
110
111
112 def design_filters():
113     designed = {}
114     for key, spec in FILTER_SPECS.items():
115         order, wn = signal.ellipord(spec["wp"], spec["ws"], RP, RS, fs=FS)
116         sos = signal.ellip(order, RP, RS, wn, btype=spec["btype"], output="sos", fs=FS)
117         designed[key] = {**spec, "order": order, "wn": wn, "sos": sos}
118     return designed
119
120
121 def response_loss_db(sos, wor_n=32768):
122     freq, response = signal.sosfreqz(sos, worN=wor_n, fs=FS)
123     magnitude = np.maximum(np.abs(response), 1e-14)
124     loss = -20 * np.log10(magnitude)
125     return freq, loss
126
127
128 def band_mask(freq, bands):
129     mask = np.zeros_like(freq, dtype=bool)
130     for low, high in bands:
131         mask |= (freq >= low) & (freq <= high)
132     return mask
133
134
135 def summarize_filter(designed):
136     lines = []
137     lines.append(f"Elliptic filter specification: rp={RP} dB, rs={RS} dB")
138     for key, spec in designed.items():
139         freq, loss = response_loss_db(spec["sos"])
140         pass_mask = band_mask(freq, spec["passbands"])
```

```

141     stop_mask = band_mask(freq, spec["stopbands"])
142     pass_loss = float(loss[pass_mask].max())
143     stop_atten = float(loss[stop_mask].min())
144     spec["pass_loss_db"] = pass_loss
145     spec["stop_atten_db"] = stop_atten
146     lines.append(
147         f"{key}: order={spec['order']}, passband max loss={pass_loss:.4f} dB, "
148         f"stopband min attenuation={stop_atten:.4f} dB"
149     )
150     return lines
151
152
153 def plot_mixed_signal():
154     t, st, _ = build_signal(N, am=False)
155     freq, mag = single_sided_spectrum(st)
156
157     fig, axes = plt.subplots(2, 1, figsize=(10.5, 6.2), constrained_layout=True)
158     time_mask = t <= 0.02
159     axes[0].plot(t[time_mask] * 1000, st[time_mask], color="C0", linewidth=1.2)
160     axes[0].set_title("Mixed SCAM signal")
161     axes[0].set_xlabel("time / ms")
162     axes[0].set_ylabel("s(t)")
163
164     freq_mask = freq <= 1200
165     markerline, stemlines, baseline = axes[1].stem(
166         freq[freq_mask],
167         mag[freq_mask],
168         basefmt=" ",
169         linefmt="C1-",
170         markerfmt="C1.",
171     )
172     plt.setp(stemlines, linewidth=0.8)
173     plt.setp(markerline, markersize=4)
174     for line in expected_lines():
175         axes[1].axvline(line, color="0.55", linewidth=0.8, linestyle="--")
176     axes[1].set_title("N-point FFT magnitude of mixed signal")
177     axes[1].set_xlabel("frequency / Hz")
178     axes[1].set_ylabel("normalized magnitude")
179     axes[1].set_xlim(0, 1200)
180     axes[1].set_ylim(0, 1.08)
181
182     path = OUT_DIR / "mixed_signal.png"
183     fig.savefig(path, bbox_inches="tight")
184     plt.close(fig)
185     return path
186
187
188 def plot_filters_loss(designed):

```

```

189 fig, axes = plt.subplots(3, 1, figsize=(10.5, 8.2), constrained_layout=True)
190 colors = ["C0", "C1", "C2"]
191 for ax, (color, (key, spec)) in zip(axes, zip(colors, designed.items())):
192     freq, loss = response_loss_db(spec["sos"])
193     ax.plot(freq, np.clip(loss, 0, 100), color=color, linewidth=1.35)
194     ax.axhline(RP, color="0.35", linestyle="--", linewidth=0.8, label="0.1 dB")
195     ax.axhline(RS, color="0.35", linestyle=":", linewidth=0.8, label="60 dB")
196     for low, high in spec["passbands"]:
197         ax.axvspan(low, high, color="C0", alpha=0.08)
198     for low, high in spec["stopbands"]:
199         ax.axvspan(low, min(high, 1200), color="C3", alpha=0.05)
200     ax.set_title(f"{spec['title']} (order {spec['order']})")
201     ax.set_xlabel("frequency / Hz")
202     ax.set_ylabel("loss / dB")
203     ax.set_xlim(0, 1200)
204     ax.set_ylim(-1, 92)
205     ax.legend(loc="upper right")
206
207 path = OUT_DIR / "filters_loss.png"
208 fig.savefig(path, bbox_inches="tight")
209 plt.close(fig)
210 return path
211
212
213 def filter_outputs(st, designed):
214     outputs = {}
215     for key, spec in designed.items():
216         outputs[spec["target"]] = signal.sosfilt(spec["sos"], st)
217     return outputs
218
219
220 def plot_separated_signals(outputs):
221     t = make_time_axis(N)
222     order = [
223         ("y3", "Low-pass output y3, fc=250 Hz"),
224         ("y2", "Band-pass output y2, fc=500 Hz"),
225         ("y1", "High-pass output y1, fc=1000 Hz"),
226     ]
227
228     fig, axes = plt.subplots(3, 2, figsize=(12, 8.6), constrained_layout=True)
229     time_mask = (t >= 0.02) & (t <= 0.08)
230
231     for row, (name, title) in enumerate(order):
232         y = outputs[name]
233         axes[row, 0].plot(t[time_mask] * 1000, y[time_mask], linewidth=1.0)
234         axes[row, 0].set_title(title)
235         axes[row, 0].set_xlabel("time / ms")
236         axes[row, 0].set_ylabel(f"{name}(n)")

```

```

237
238     freq, mag = single_sided_spectrum(y)
239     freq_mask = freq <= 1200
240     axes[row, 1].plot(freq[freq_mask], mag[freq_mask], linewidth=1.25, color=f"C{row + 1}")
241     axes[row, 1].set_title(f"Spectrum of {name}(n)")
242     axes[row, 1].set_xlabel("frequency / Hz")
243     axes[row, 1].set_ylabel("normalized magnitude")
244     axes[row, 1].set_xlim(0, 1200)
245     axes[row, 1].set_ylim(0, 1.08)
246
247     path = OUT_DIR / "separated_signals.png"
248     fig.savefig(path, bbox_inches="tight")
249     plt.close(fig)
250     return path
251
252
253 def plot_fft_length_comparison():
254     lengths = [1600, 1800, 2000]
255     fig, axes = plt.subplots(3, 1, figsize=(10.5, 8.0), constrained_layout=True)
256     checks = []
257     for ax, n_samples in zip(axes, lengths):
258         _, st, _ = build_signal(n_samples, am=False)
259         freq, mag = single_sided_spectrum(st, nfft=n_samples)
260         freq_mask = freq <= 1200
261         ax.plot(freq[freq_mask], mag[freq_mask], linewidth=1.05)
262         for line in expected_lines():
263             ax.axvline(line, color="0.55", linewidth=0.8, linestyle="--")
264         bin_spacing = FS / n_samples
265         integer_bins = [abs(line * n_samples / FS - round(line * n_samples / FS)) < 1e-9 for line in
266                        expected_lines()]
267         all_ideal = all(integer_bins)
268         non_integer = [line for line, ok in zip(expected_lines(), integer_bins) if not ok]
269         checks.append((n_samples, bin_spacing, all_ideal, non_integer))
270         ax.set_title(
271             f"N={n_samples}, bin spacing={bin_spacing:.4f} Hz, "
272             f"all six lines on DFT bins: {all_ideal}"
273         )
274         ax.set_xlabel("frequency / Hz")
275         ax.set_ylabel("normalized magnitude")
276         ax.set_xlim(0, 1200)
277         ax.set_ylim(0, 1.08)
278
279     path = OUT_DIR / "fft_length_comparison.png"
280     fig.savefig(path, bbox_inches="tight")
281     plt.close(fig)
282     return path, checks
283

```

```
284 def plot_am_vs_scam():
285     t, scam, _ = build_signal(N, am=False)
286     _, am_sig, _ = build_signal(N, am=True)
287     scam_freq, scam_mag = single_sided_spectrum(scam)
288     am_freq, am_mag = single_sided_spectrum(am_sig)
289
290     fig, axes = plt.subplots(2, 2, figsize=(12, 7.0), constrained_layout=True)
291     time_mask = t <= 0.03
292     axes[0, 0].plot(t[time_mask] * 1000, scam[time_mask], linewidth=1.0)
293     axes[0, 0].set_title("SCAM mixed signal")
294     axes[0, 0].set_xlabel("time / ms")
295     axes[0, 0].set_ylabel("amplitude")
296
297     axes[0, 1].plot(t[time_mask] * 1000, am_sig[time_mask], color="C1", linewidth=1.0)
298     axes[0, 1].set_title("AM mixed signal, A0=Am=1")
299     axes[0, 1].set_xlabel("time / ms")
300     axes[0, 1].set_ylabel("amplitude")
301
302     freq_mask = scam_freq <= 1200
303     axes[1, 0].stem(
304         scam_freq[freq_mask],
305         scam_mag[freq_mask],
306         basefmt=" ",
307         linefmt="C0-",
308         markerfmt="C0.",
309     )
310     axes[1, 0].set_title("SCAM spectrum")
311     axes[1, 0].set_xlabel("frequency / Hz")
312     axes[1, 0].set_ylabel("normalized magnitude")
313     axes[1, 0].set_xlim(0, 1200)
314     axes[1, 0].set_ylim(0, 1.08)
315
316     freq_mask = am_freq <= 1200
317     axes[1, 1].stem(
318         am_freq[freq_mask],
319         am_mag[freq_mask],
320         basefmt=" ",
321         linefmt="C1-",
322         markerfmt="C1.",
323     )
324     for carrier in [250, 500, 1000]:
325         axes[1, 1].axvline(carrier, color="0.55", linewidth=0.8, linestyle="--")
326     axes[1, 1].set_title("AM spectrum with carrier components")
327     axes[1, 1].set_xlabel("frequency / Hz")
328     axes[1, 1].set_ylabel("normalized magnitude")
329     axes[1, 1].set_xlim(0, 1200)
330     axes[1, 1].set_ylim(0, 1.08)
331
```



```
332     path = OUT_DIR / "am_vs_scam.png"
333     fig.savefig(path, bbox_inches="tight")
334     plt.close(fig)
335     return path
336
337
338 def spectral_line_summary(st):
339     freq, mag = single_sided_spectrum(st, normalize=True)
340     lines = ["Expected positive-frequency SCAM lines:"]
341     for line in expected_lines():
342         idx = int(np.argmin(np.abs(freq - line)))
343         lines.append(f"{line:7.1f} Hz -> nearest DFT bin {freq[idx]:7.2f} Hz, normalized magnitude {
344             mag[idx]:.4f}")
345     return lines
346
347 def write_summary(designed, fft_checks, figure_paths, st):
348     summary = []
349     summary.extend(summarize_filter(designed))
350     summary.append("")
351     summary.extend(spectral_line_summary(st))
352     summary.append("")
353     summary.append("FFT length ideal-line check:")
354     for n_samples, bin_spacing, all_ideal, non_integer in fft_checks:
355         if non_integer:
356             missing = ", ".join(f"{item:g} Hz" for item in non_integer)
357         else:
358             missing = "none"
359         summary.append(
360             f"N={n_samples}: bin spacing={bin_spacing:.6f} Hz, all ideal={all_ideal}, "
361             f"non-integer-bin lines={missing}"
362         )
363     summary.append("")
364     summary.append("Generated figures:")
365     for path in figure_paths:
366         summary.append(f"- {path.as_posix()}")
367
368     summary_path = OUT_DIR / "verification_summary.txt"
369     summary_path.write_text("\n".join(summary) + "\n", encoding="utf-8")
370     return summary_path, summary
371
372
373 def main():
374     OUT_DIR.mkdir(parents=True, exist_ok=True)
375     set_common_style()
376
377     t, st, _ = build_signal(N, am=False)
378     designed = design_filters()
```

```
379     outputs = filter_outputs(st, designed)
380
381     figure_paths = [
382         plot_mixed_signal(),
383         plot_filters_loss(designed),
384         plot_separated_signals(outputs),
385     ]
386     fft_path, fft_checks = plot_fft_length_comparison()
387     figure_paths.append(fft_path)
388     figure_paths.append(plot_am_vs_scam())
389
390     summary_path, summary = write_summary(designed, fft_checks, figure_paths, st)
391
392     print("\n".join(summary))
393     print(f"\nSummary written to: {summary_path}")
394
395
396 if __name__ == "__main__":
397     main()
```

代码 A.1 实验二 Python 程序